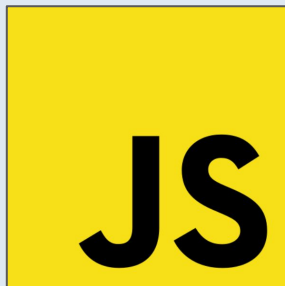


Mòdul 06



Institut TIC
de Barcelona



Desenvolupament web en entorn
client

RA7.1: Comunicació asíncrona client-servidor.

1. Tipus de funcions
2. Callbacks
3. Javascript asíncron
4. Promises
5. Fetch
6. Async/Await

Function declaration

- Li definim un nom a la funció.
- Es poden cridar abans de declarar-la.

```
function calcularEdat(dataNaixement) {  
  return 2024 - dataNaixement;  
}
```

Function expression

- Funció que es guarda en una variable.
- Normalment s'utilitzen amb **funcions anònimes (funcions sense nom)**.
- No es poden cridar abans de declarar-la.

```
const calcularEdat = function (dataNaixement) {  
  return 2024 - dataNaixement;  
};
```

Les 2 funcions es criden igual i donen el mateix resultat:

```
calcularEdat(1990);
```

Arrow function

- Funció que es guarda en una variable.
- S'utilitzen amb **funcions anònimes**
- No es poden cridar abans de declarar-la.
- Funció compactada (intenta simplificar la sintaxis). `const hello = () => console.log("hello world");`
- **No es recomana** utilitzar-la **per definir objectes** ja que no pot utilitzar el *this* del mateix scope
- Ideal per a **funcions petits i simples i callbacks** (funcions que criden altres funcions)

```
const calcularEdat = (any) => {  
  const edat = 2026 - any;  
  return edat;  
}
```

```
const calcularEdat = (any) => 2026 - any;
```

Es criden igual que les altres funcions:

```
calcularEdat(1990);
```

- Són funcions (*expression* o *arrow*) que es passen com a paràmetres d'altres funcions.
- Molt aconsellable utilitzar les arrow function.
- Molt útil per gestionar l'execució de les funcions.

```
const funcioPrincipal = function(myCallback){  
  myCallback();  
}  
  
const funcioParam = () => {  
  console.log("Executo la funció per paràmetre");  
}  
  
funcioPrincipal(funcioParam);
```

Executo la funció per paràmetre

Funció que rep per paràmetre una altra funció.

Funció que enviarem com a paràmetre dins de la funció *funcioPrincipal*.

Executem la funció principal i com a paràmetre li passem **la funció com una constant**

- Algunes consideracions importants

```
const funcioPrincipal = function (myCallback){  
  myCallback();  
}  
  
const funcioParam = () => {  
  console.log("Executo la funció per paràmetre");  
}  
  
funcioPrincipal(funcioParam());
```

COMPTE!! S'ha de passar la funció com una constant! Si li posem () l'estem executant.

```
const funcioPrincipal = function (myCallback){  
  myCallback();  
}  
  
const funcioParam = (text) => {  
  console.log(text);  
}  
  
funcioPrincipal(() => funcioParam(text));
```

Si li hem de passar paràmetres, hem de definir un funció anònima que encapsuli la funcióParam

- Molt habitual per utilitzar en mètodes com: *addEventListener*, *setTimeout*, *setInterval*

```
const funcioParametre = function () {  
  console.log("Executo la funció paràmetre");  
};  
  
setTimeout(funcioParametre, 3000);
```

```
const funcioParametre = function (parametre) {  
  console.log(  
    "Executo la funció paràmetre amb un paràmetre " + parametre  
  );  
};  
  
setTimeout(() => funcioParametre("text"), 3000);
```

+

El resultat és:

```
const objButton1 = document.querySelector("#btn1");
const objButton2 = document.querySelector("#btn2");
const objButton3 = document.querySelector("#btn3");

const objNum1 = document.querySelector("#num1");
const objNum2 = document.querySelector("#num2");
const objResult = document.querySelector("#divResult");
// FUNCIO PRINCIPAL
const operar = function (myFunction){
    const result = Number(objNum1.value) + Number(objNum2.value);
    myFunction(result);
}
//FUNCTIONS ARROW PER FER CALLBACKS
const mostrarLog = (result) => console.log(result);
const mostrarAlert = (result) => alert(result);
const mostrarPantall = (result) => objResult.textContent = result;

objButton1.addEventListener("click", () =>{
    operar(mostrarLog);
});

objButton2.addEventListener("click", () =>{
    operar(mostrarAlert);
});

objButton3.addEventListener("click", () =>{
    operar(mostrarPantall);
});
```


Execució del codi

El codi generalment és síncron:

- S'executa línia per línia.
- Fins que no acaba una línia no executa la següent.
- Inclús amb les funcions compleix l'ordre d'execució per línies per on es criden i no per on es declaren.

```
console.log("primera línia");  
console.log("segona línia");  
console.log("tercera línia");
```

primera línia

segona línia

tercera línia

```
console.log("primera línia");  
console.log("segona línia");  
funcio();  
console.log("tercera línia");  
  
function funcio() {  
  console.log("línia dins de la funció");  
}
```

primera línia

segona línia

línia dins de la funció

tercera línia

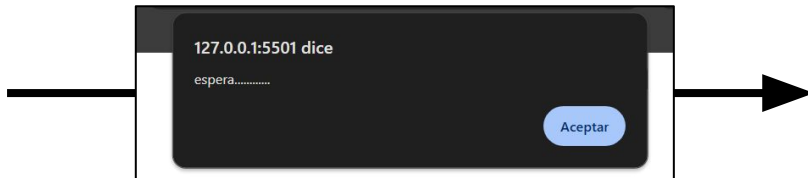
Programació síncrona

El problema de la programació síncrona és que hi pot haver operacions de llarga durada que **bloquegen** l'execució i fins que no acaben no es pot continuar avançant.

```
console.log("primera línia");  
console.log("segona línia");  
funcioEspera();  
console.log("tercera línia");  
  
function funcioEspera() {  
  alert("espera.....");  
  console.log("línia dins de la funció");  
}
```

↓ Bloqueja l'execució del codi

```
primera línia  
segona línia
```



```
primera línia  
segona línia  
línia dins de la funció  
tercera línia
```

Mentres l'usuari no accepti l'alerta el codi queda bloquejat i no s'executarà les línies que hi ha després.

Programació asíncrona

Permet continuar executant el codi sense esperar que una operació de llarga durada hagi d'acabar.

El codi que bloqueja treballa en background (un altre thread) mentre el thread principal pot continuar executant el codi del programa.

Exemples de funcionalitats asíncrones:

- *setTimeout*
- *setInterval*
- *Càrrega de fitxers*
- *API Fetch (peticions http)*
- *API Geolocation*

```
console.log("primera línia");  
console.log("segona línia");  
setTimeout(funcioEspera, 3000);  
console.log("tercera línia");  
  
function funcioEspera() {  
    alert("espera.....");  
    console.log("línia dins de la funció");  
}
```

setInterval

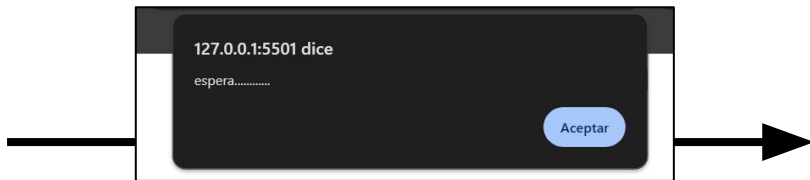
```
const id = setInterval(callback, delay);
```

- callback → la funció que s'executa cada vegada
 - delay → temps en mil·lisegons que torna a executar la funció
 - id → identificador
-
- clearInterval(id) → permet aturar l'interval d'execució mitjançant l'identificador

Programació asíncrona

```
console.log("primera línia");  
console.log("segona línia");  
setTimeout(funcioEspera, 3000);  
console.log("tercera línia");  
  
function funcioEspera() {  
    alert("espera.....");  
    console.log("línia dins de la funció");  
}
```

```
primera línia  
segona línia  
tercera línia
```



```
primera línia  
segona línia  
tercera línia  
línia dins de la funció
```

La funció `setTimeout` fa que el codi continui executant-se sense que es bloquegi durant els 3 segons que tarda en cridar la funció `funcioEspera` (i mostra “tercera línia” en tercera posició). Un cop passat els 3 segons mostra en última posició “línia dins de la funció”.

Ordre d'execució

Programació síncrona

```
console.log("primera línia");  
console.log("segona línia");  
funcioEspera();  
console.log("tercera línia");  
  
function funcioEspera() {  
  alert("espera.....");  
  console.log("línia dins de la funció");  
}
```



```
primera línia  
segona línia  
línia dins de la funció  
tercera línia
```

Programació asíncrona

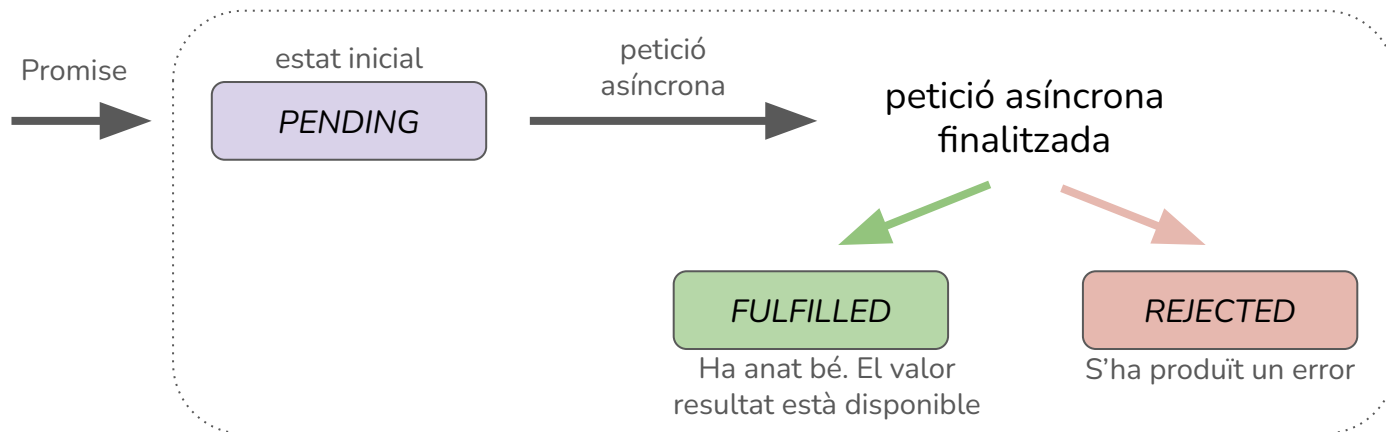
```
console.log("primera línia");  
console.log("segona línia");  
setTimeout(funcioEspera, 3000);  
console.log("tercera línia");  
  
function funcioEspera() {  
  alert("espera.....");  
  console.log("línia dins de la funció");  
}
```



```
primera línia  
segona línia  
tercera línia  
línia dins de la funció
```

- Objecte que controla la finalització d'una petició asíncrona i serveix com a contenidor del seu valor resultant.
- Intenta donar resposta a les peticions que no sabem quan acabaran d'una forma que sigui fàcil de gestionar, controlar o encadenar amb altres peticions asíncrones.
- Té 3 estats: *pending*, *fulfilled* i *rejected*.

- Cicle de vida i estats d'una promesa:



Promise.state	Promise.result
pending	undefined
fulfilled	a result value
rejected	an error object

- Sintaxis - Crear promesa

```
let promise = new Promise((resolve, reject)=>{});
```

```
let myPromise = new Promise(function (resolve, reject) {  
  let x = 0;  
  if (x == 0) {  
    resolve("OK");  
  } else {  
    reject("Error");  
  }  
});
```

Creem la promesa amb els paràmetres *resolve* i *reject*

Utilitzem el paràmetre *resolve* com a funció per indicar que la promesa s'ha resolt correctament

Utilitzem el paràmetre *reject* com a funció per indicar que a la promesa s'ha produït un error.

- Mètodes
 - `then()` → Ens tornarà el valor de resolve
 - `catch()` → Ens tornarà el valor de reject
 - `finally()` → S'executarà una vegada s'hagi executat el `then` o el `catch`, ens pot servir per amagar una animació de loading, mostrar un missatge a l'usuari...
 - `all()` → Li passarem un array de promeses i les analitzarà, a resolve si totes han anat bé ens tornarà un array amb els resultats de cada una, si alguna falla ens tornarà el valor de reject.
 - `race()` → Li passem un array de promeses i les analitzarà en funció de la seva rapidesa

- Sintaxis - Recuperar promesa

```
myPromise
  .then(function (value) {
    console.log(value);
  })
  .catch(function (value) {
    console.log(value);
  });
```

THEN Recuperar la informació d'una promesa que ha anat bé.

CATCH Recupera la informació d'una promesa que ha anat malament.



El mateix codi però amb arrow functions

```
myPromise
  .then((value) => console.log(value))
  .catch((value) => console.log(value));
```

Una altre forma de fer-ho seria que en el then la primera funció serà el resolve i la segona el reject.

```
myPromise.then(
  function (value) {
    console.log(value);
  },
  function (value) {
    console.log(value);
  }
);
```

- all

```
let p1 = new Promise((resolve, reject) =>
  | setTimeout(resolve("P1"), 1000)
);

let p2 = new Promise((resolve, reject) =>
  | setTimeout(resolve("P2"), 2000)
);

let p3 = new Promise((resolve, reject) =>
  | setTimeout(resolve("P3"), 3000)
);

Promise.all([p1, p2, p3])
  .then((values) => console.log(values))
  .catch((reason) => console.log(reason));
```

Li passarem un array de promeses i les analitzarà, si totes han anat bé ens tornarà un array amb els resultats de cada una, si alguna falla ens tornarà el valor de reject,

- Afegir en el vostre joc un compte enrere de 2 minuts que a l'arribar a 0 doni la partida com a perduda. Ha de mostrar la paraula, ha de sortir el fons de color vermell i es fa els càlculs del total partides.

- S'ha de fer servir el `setInterval` i `clearInterval`

```
const minuts = Math.floor( tempsTotal / 60);  
const segons = tempsTotal % 60;  
const display = `${minuts.toString().padStart(2, '0')}:${segons.toString().padStart(2, '0')}`;
```

```
fetch("url")  
  .then(function (respons) {  
    return respons.json();  
  })  
  .then(function (data) {  
    console.log(data);  
  });
```

FETCH

Permet indicar una *url* per poder fer una petició ***request*** a través del protocol http.

Retorna una promise

RESPOSTA

Bloc de codi que serveix per obtenir i tractar la informació de la resposta del fetch.

Sempre ha de retornar un valor.

Retorna una promise.

DADES

Bloc de codi que serveix per obtenir la informació que retorna la API.
Generalment la informació està estructurada a través d'un json.

```
const peticio = fetch(url);  
console.log(peticio);
```

```
▼ Promise { <state>: "pending" }  
  <state>: "fulfilled"  
  ► <value>: Response { type: "basic",
```

El *fetch* retorna una promesa

PROMISE 1

state

value

El *fetch* retorna una promesa que conté *state* i *value*.
No puc recuperar directament el *value* d'aquesta promesa.
Haig d'utilitzar **then** per poder-la obtenir.

```
const peticio = fetch(url);  
console.log(peticio);
```

```
▼ Promise { <state>: "pending" }  
  <state>: "fulfilled"  
  ▶ <value>: Response { type: "basic",
```

El **fetch** retorna una promesa

PROMISE 1

state

value

Amb el **then** recupero el **value** de la promesa 1.
La puc obtenir amb el **paràmetre de la funció**.

```
const reposta = peticio.then(function(valuePromisePeticio){  
  return valuePromisePeticio.json();  
});  
console.log(reposta);
```

El **then** retorna una promesa

PROMISE 2

state

value

Dins un **then**, el **return** retorna el **value** de la nova promesa

```
▼ Promise { <state>: "pending" }  
  <state>: "fulfilled"  
  ▶ <value>: Object { alfabet: (28) [...] }
```

El **then** retorna una nova promesa (promise 2).
El **return** informa del valor que tindrà el **value** de la promesa 2.
No puc recuperar el value directament -> haig d'utilitzar un nou **then**


```
const peticio = fetch(url);  
console.log(peticio);
```

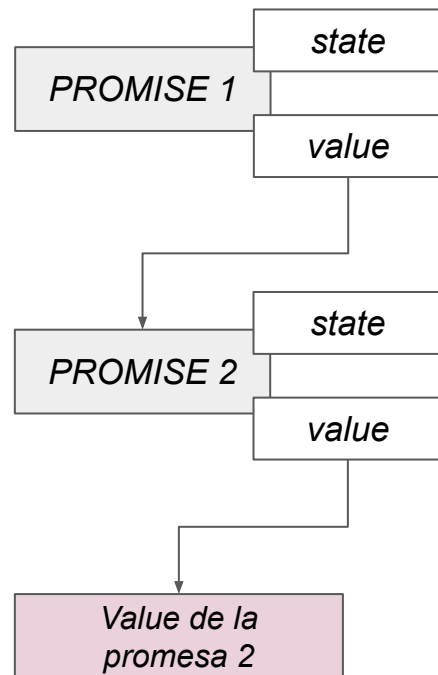
```
const reposta = peticio.then(function(valuePromisePeticio){  
  return valuePromisePeticio.json();  
});  
console.log(reposta);
```

▼ Promise { <state>: "pending" }
 <state>: "fulfilled"
 ► <value>: Object { alfabet: (28) [...] }

Amb el **then** recupero el **value** de la promesa 2.
La puc obtenir amb el **paràmetre de la funció**.

```
const dadaFinal = reposta.then(function (valuePromiseResposta){  
  console.log(valuePromiseResposta);  
})
```

▼ Object { alfabet: (28) [...] }
 ► alfabet: Array(28) ["A", "B", "C", ...]



```
fetch("http://127.0.0.1:5501/alfabet.json")
  .then(function (resposta) {
    // Resposta del fetch
    console.log(resposta);
    //Sempre ha de tenir un return...
    return resposta.json();
  })
  .then(function (dades) {
    // Dada que obtenim de la resposta.
    // Es retorna amb format Json.
    console.log(dades);
  })
  .catch(function (error) {
    //Per controlar que no hi hagi errors
    //durant el procés de la consulta
    console.error(error);
  });
```

Si durant el procés de la consulta hi ha un error no controlat, tant en el *fetch* com en qualsevol dels *then*, el podem gestionar a través del *catch*.

Els errors més habituals són: problemes de connexió, timeouts o bugs de programació a l'entorn client.

La funció retorna una paràmetre d'entrada per poder controlar l'error: missatge, stack,...

```
console.log(error);
//Sempre ha de tenir un return...
return resposta.json();
})
.then(function (dades) {
  // Dada que obtenim de la resposta.
  // Es retorna amb format Json.
  console.log(dades);
})
.catch(function (error) {
  //Per controlar que no hi hagi errors
  //durant el procés de la consulta
  console.log(error);
});
```

TypeError

```
columnNumber: 1
fileName: ""
lineNumber: 0
message: "NetworkError when attempting to fetch resource."
stack: ""
__proto__: TypeError.prototype
```

error: TypeError: NetworkError when attempting to fetch resource.

```
fetch("http://127.0.0.1:5501/alfabet.json" )
  .then(function (resposta) {
    // Resposta del fetch
    console.log(resposta.ok, resposta.status);
    if (!resposta.ok) {
      console.log(`Error inesperat: ${resposta.status}`);
      throw new Error(`Error inesperat: ${resposta.status}`);
    } else {
      console.log("La consulta és correcta");
    }
    //Sempre ha de tenir un return...
    return resposta.json();
  })
  .then(function (dades) {
    // Dada que obtenim de la respota.
    // Es retorna amb format Json.
    console.log(dades);
  })
  .catch(function (error) {
    //Per controlar que no hi hagi errors
    //durant el procés de la consulta
    console.error(error);
  });
```

Les peticions fetch es caracteritzen generalment per **demanar informació a sistemes de tercers**.

Com que **no tenim cap control sobre aquests sistemes**, per tal de poder conèixer i gestionar l'estat final de la consulta, una pràctica molt habitual és **utilitzar les propietats `ok` o `status` de la resposta** per tal de poder gestionar els errors externs dins de l'entorn client.

```
▼ Response { type: "basic", url: "http://127.0.0.1:5501/alfabet.json" }
  ▶ body: ReadableStream { locked: true }
  bodyUsed: true
  ▶ headers: Headers(11) { "accept-ranges" → "bytes", "cache-control" → "no-cache", "content-type" → "application/json", "date" → "Wed, 07 Nov 2024 14:00:00 GMT", "etag" → "W/\"1234567890\""}
    ok: true
    redirected: false
    status: 200
    statusText: "OK"
    type: "basic"
    url: "http://127.0.0.1:5501/alfabet.json"
```

Exemple [status](#)

```
false 404
```

```
Error inesperat: 404
```

```
! ▶ "Error controlat en el catch:Error: Error inesperat: 404"
```

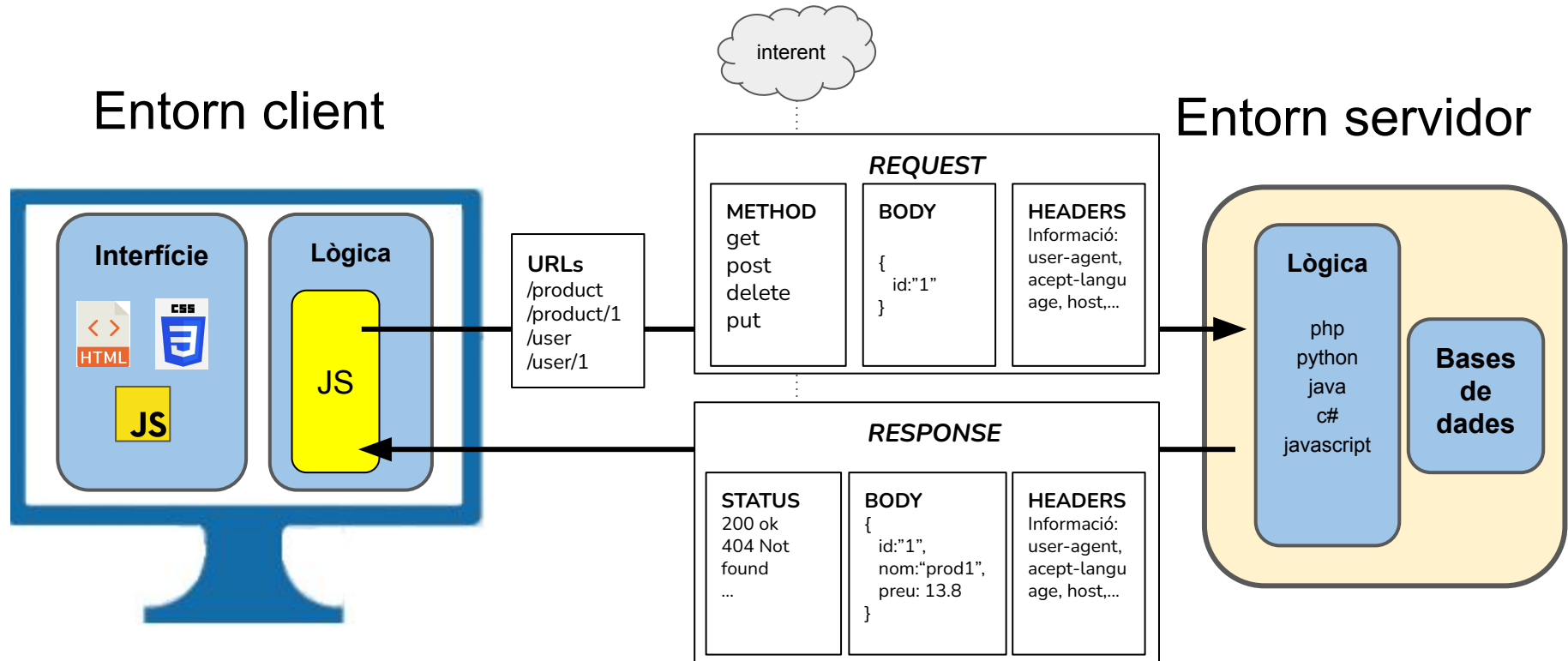
```
fetch("http://127.0.0.1:5501/alfabet.json" )  
  .then(function (resposta) {  
    // Resposta del fetch  
    console.log(resposta);  
    //Sempre ha de tenir un return...  
    return resposta.json();  
  })  
  .then(function (dades) {  
    // Dada que obtenim de la respota.  
    // Es retorna amb format Json.  
    console.log(dades);  
  })  
  .catch(function (error) {  
    //Per controlar que no hi hagi errors  
    //durant el procés de la consulta  
    console.log(error);  
  })  
  .finally(function () {  
    //Últim codi que sempre s'executarà al final del fetch  
    //tan si ha anat tot bé com si hi ha algun error.  
    console.log("Codi final del fetch" );  
  });
```

Codi que s'executarà **sempre** al final del fetch independentment de si ha anat bé o hi ha hagut algun error.

```
► Response { type: "basic", url: "http
```

```
► Object { alfabet: (26) [...] }
```

```
Codi final del fetch
```



```
const response = await fetch(url, {  
  method: "POST",  
  headers: {  
    Authorization: `Bearer ${token}`  
  },  
  body: JSON.stringify({  
    uris: [trackUri], // Afegir la informació que  
volem tractar  
  }),  
});
```

Exemple de POST

```
const response = await fetch(url, {  
  method: "DELETE",  
  headers: {  
    Authorization: `Bearer ${token}`  
  },  
  body: JSON.stringify({  
    tracks: [{ uri: trackUri }],  
  }),  
});
```

Exemple de DELETE

```
const response = await fetch(url, {  
  method: "PUT",  
  headers: {  
    Authorization: `Bearer ${token}`  
  },  
  body: JSON.stringify({ name: "Nom llista personal" }),  
});
```

Exemple de PUT


```
//Hem de definir la funció com async
async function getAlfabet() {
  const url = "http://127.0.0.1:5502/alfabet.json"
  try {
    //Fem la consulta del fetch amb await.
    const response = await fetch(url, { method: "GET" });
    //Validem que la consulta hagi anat bé.
    if (response.ok) {
      console.log("La consulta ha anat bé");
    } else {
      console.log("La consulta ha tingut algun error");
      //Llancem una excepció que controlarà el catch
      throw new Error(
        `Error: ${response.status} - ${response.statusText}`
      );
    }
    //Obtenim la informació del resultat a través del json();
    const data = await response.json();
    if (data) {
      console.log("Hi ha informació");
    } else {
      console.log("No hi ha informació.");
    }
  } catch (error) {
    console.log("Error durant la consulta de la informació" error);
  } finally {
    console.log("Informació final de la consulta");
  }
}
```

La funció sempre ha de ser **async**

La propietat **await** permet que dins una funció **async** poguem fer peticions asíncrones. En aquest cas per tornar la informació de la petició.

No arribarem a aquest codi fins que la **const response** no tingui la informació.

La propietat **await** permet que dins una funció **async** poguem fer peticions asíncrones. En aquest cas per tornar la informació en format **json** de les dades.

(then) vs (async / await)

```
const getAlfabet = function () {  
  fetch("http://127.0.0.1:5501/alfabet.json")  
  .then(function (resposta) {  
    console.log(resposta);  
    if(resposta.ok)  
    {  
      console.log("La resposta és ok");  
      return resposta.json();  
    }  
    else{  
      console.log("Error de la resposta");  
      throw new Error("Error");  
    }  
  })  
  .then(function (dades) {  
    console.log("Obtenim les dades", dades);  
  })  
  .catch(function (error) {  
    console.log(error);  
  })  
  .finally(function () {  
    console.log("Codi final del fetch");  
  });  
};
```

```
async function getAlfabet(id) {  
  const url = "http://127.0.0.1:5502/alfabet.json";  
  
  try {  
    const resposta = await fetch(url, { method: "GET" });  
  
    if (resposta.ok) {  
      console.log("La resposta és ok");  
    } else {  
      console.log("Error en la resposta");  
      throw new Error("Error en la resposta");  
    }  
  
    const dades = await resposta.json();  
    console.log("Obtenim les dades", dades);  
  } catch (error) {  
    console.log(error);  
  } finally {  
    console.log("Codi final del fetch");  
  }  
}
```

```
fetch("http://127.0.0.1:5500/res/alfabet_ca.json")
  .then(function (response) {
    return response.json();
  })
  .then(function(data) {
    alfabet = data.alfabet;
    console.log("info dins del then" + alfabet);
    for(let i = 0; i< alfabet.length; i++){
      const boto = document.createElement("button");
      boto.className = "button-game";
      boto.textContent = alfabet[i];
      boto.addEventListener("click", () => jugar(boto));
      seccioBotons.appendChild(boto);
    }
  });
```

```
const URL_POKEEMON = "https://pokeapi.co/api/v2/pokemon/pikachu" ;
const imgObj = document.querySelector (".imatge");
let srcImg = "";
function getImg () {

    console.log("Linia 11");

    fetch(URL_POKEEMON)
    .then(function (resposta) {
        if (!resposta.ok) {
            throw Error ("La petició no ha anat bé" );
        }
        return resposta.json ();
    })
    .then(function (dada) {
        console.log(dada.sprites.front_shiny);
        srcImg = dada.sprites.front_shiny;
        imgObj.src = srcImg;
    })
    .catch(function (error) {
        console.log("Hi hagut un error" );
    })
    .finally(function () {
        console.log("Sempre s'executa" );
    });

    console.log("Linia 44");
    console.log(srcImg);
}
getImg ();
```